

COMPLETE PROJECT PLAN

FAKE JOB POSTING DETECTION SYSTEM

Including Social Media Ad Detection (Facebook / Instagram / Meta)

**Team Size 4
Members**

Duration 5 Weeks

**Tech Stack Python +
React + FastAPI**

**Target 99%+
Accuracy**

What this document covers:

■ Complete folder & file
structure

■■ Environment setup —
every command

■ ML model — step by
step code

■ FastAPI backend — full
implementation

■ React frontend —
complete UI

■ Deployment on Render
+ Vercel

■ Testing strategy & test
cases

■ Documentation &
presentation

TABLE OF CONTENTS

1. Project Overview & Problem Statement	3
2. System Architecture — Full Diagram	3
3. Tech Stack — Every Tool Explained	4
4. Complete Folder & File Structure	4
5. PHASE 1 — Environment Setup	5
5.1 Install Python, Node.js, Git	5
5.2 Create virtual environment	5
5.3 Install all Python packages	5
5.4 Setup project folders	5
6. PHASE 2 — Dataset & Data Preparation	6
6.1 Download EMSCAD Kaggle dataset	6
6.2 Exploratory Data Analysis (EDA)	6
6.3 Data Cleaning & Preprocessing	7
6.4 Handle Class Imbalance (ADASYN)	7
7. PHASE 3 — Machine Learning Model	8
7.1 Feature Engineering	8
7.2 Train Baseline Model (TF-IDF + RF)	8
7.3 Train BERT Classifier	9
7.4 Add SHAP Explainability	10
7.5 Save & Export Model	10
8. PHASE 4 — URL Checker Module	11
9. PHASE 5 — Image Analyzer Module	12
10. PHASE 6 — FastAPI Backend	13
10.1 Project structure & setup	13
10.2 All API endpoints — full code	13
10.3 Risk scoring logic	15
11. PHASE 7 — React Frontend	16
11.1 Create React App & install dependencies	16
11.2 Pages & Components — full code	16
11.3 API integration	18
12. PHASE 8 — Integration & Testing	19
13. PHASE 9 — Deployment	20
13.1 Deploy backend on Render	20
13.2 Deploy frontend on Vercel	20
14. PHASE 10 — Documentation & Presentation	21

15. Team Task Assignments — All Phases 22

16. Common Errors & How to Fix Them 23

1. PROJECT OVERVIEW & PROBLEM STATEMENT

What Are We Building?

We are building a **Fake Job Posting Detection System** — a web application that uses Artificial Intelligence (Machine Learning + NLP + Computer Vision) to automatically detect whether a job posting is real or fake. The system works on **both job board posts** (LinkedIn, Naukri, Internshala) and **social media job ads** (Facebook, Instagram, Meta Ads). A user can paste job text, upload an ad screenshot, or enter a URL — and the system returns a **Risk Score (0–100)** with a clear explanation of why the post was flagged.

The Problem In Numbers

The Problem	Scale & Impact
Fake job postings on LinkedIn, Naukri, Indeed, Facebook, Instagram	Millions of fake posts go live every month globally
Job seekers lose personal data (Aadhaar, bank details) to scammers	Financial loss + identity theft affecting crores in India
No existing tool detects fakes across BOTH job boards + social media	Our project fills this exact gap
Manual moderation is too slow — impossible at scale	AI automation is the only viable solution

What Our System Does — 3 Input Types

Input Type	What User Does	What System Analyzes	Output
Text Input	Paste job post text	NLP + BERT classifier on text content	Risk Score + Reason Cards
Image Upload	Upload ad screenshot	CLIP vision model — logos, visuals, text	Suspicious Visual Flags
URL Input	Paste job/ad URL	Domain age, SSL, Safe Browsing API, SafeDNS	Safe/Unsafe URL flag

2. SYSTEM ARCHITECTURE — HOW EVERYTHING CONNECTS

Full System Data Flow

The architecture has 5 layers. Every user request flows through all layers and returns a combined risk score with explanation.

LAYER 1: USER INPUT	React Frontend User enters text / uploads image / pastes URL on the React web app
LAYER 2: API GATEWAY	FastAPI Backend FastAPI receives the request, validates input, calls the right modules

LAYER 3: AI MODULES	3 Parallel Analyzers Module A: Text NLP (BERT) Module B: Image (CLIP) Module C: URL Checker
LAYER 4: RISK SCORER	Combined Score Engine Combines all 3 module outputs into 1 risk score (0-100) with SHAP reasons
LAYER 5: RESPONSE	JSON → React UI Risk score, label (Safe/Suspicious/Scam), reason cards sent back to frontend

Meta Ad Library API Integration

For social media ads specifically, the backend also calls the **Meta Ad Library API** (completely free and public — no login needed) to fetch ad metadata: how long the ad has been running, advertiser page creation date, spending history, and targeted regions. These signals are added to the risk score. Suspicious signals: ad running less than 3 days, new advertiser page, targeting only low-income regions, no prior ad history.

3. COMPLETE TECH STACK — EVERY TOOL EXPLAINED

Layer	Tool / Library	Version	Why We Use It	Who Uses It
ML — Text	scikit-learn	1.3+	TF-IDF vectorizer + Random Forest baseline model	P1
ML — Text	transformers (HuggingFace)	4.35+	BERT fine-tuning for advanced text classification	P1
ML — Text	torch (PyTorch)	2.0+	Deep learning backend for BERT model training	P1
ML — Text	shap	0.43+	Explainability — shows WHY a post was flagged	P1
ML — Image	transformers (CLIP)	4.35+	Vision model to analyze job ad images and logos	P2
ML — Image	Pillow (PIL)	10+	Image loading, resizing, preprocessing	P2
URL Check	requests	2.31+	HTTP calls to Google Safe Browsing API	P2
URL Check	python-whois	0.8+	Check domain age and registration details	P2
Data	pandas	2.0+	Load, clean, and manipulate the dataset	P1+P2
Data	numpy	1.24+	Numerical operations on arrays	P1+P2
Data	imbalanced-learn	0.11+	ADASYN oversampling to fix class imbalance	P1
Backend	fastapi	0.104+	High-performance async API framework for ML service	P3
Backend	uvicorn	0.24+	ASGI server to run FastAPI application	P3
Backend	python-multipart	0.0.6+	Handle file uploads (images) in FastAPI	P3
Backend	pydantic	2.0+	Data validation for API request/response models	P3
Frontend	React	18+	Component-based UI framework for the web app	P4
Frontend	Axios	1.6+	HTTP client to call FastAPI endpoints from React	P4
Frontend	TailwindCSS	3+	Utility-first CSS for styling components	P4
Frontend	react-router-dom	6+	Client-side routing between pages	P4
Deployment	Render	-	Cloud platform to deploy FastAPI backend (free tier)	P3
Deployment	Vercel	-	Static hosting for React frontend (free tier)	P4
Dev Tools	Git + GitHub	-	Version control and code collaboration	All
Dev Tools	Postman	-	Test API endpoints before frontend is ready	P3
Dev Tools	VS Code	-	Code editor (install Python + ESLint extensions)	All

4. COMPLETE FOLDER & FILE STRUCTURE

Create this exact folder structure on your computer before writing any code. Every file listed below will be created and explained in this document.

```
fake-job-detector/
■
■ ■ ■ ■ backend/ ← FastAPI Python backend
■ ■ ■ ■ main.py ← FastAPI app entry point
■ ■ ■ ■ requirements.txt ← All Python dependencies
■ ■ ■ ■ config.py ← API keys & config variables
■ ■ ■ ■ routers/
■ ■ ■ ■ analyze.py ← /analyze endpoints
■ ■ ■ ■ health.py ← /health check endpoint
■ ■ ■ ■ services/
■ ■ ■ ■ text_analyzer.py ← BERT text classification
■ ■ ■ ■ image_analyzer.py ← CLIP image analysis
■ ■ ■ ■ url_checker.py ← URL safety check
■ ■ ■ ■ meta_api.py ← Meta Ad Library API calls
■ ■ ■ ■ risk_scorer.py ← Combine all signals → score
■ ■ ■ ■ models/
■ ■ ■ ■ bert_model/ ← Saved fine-tuned BERT model
■ ■ ■ ■ tfidf_rf_model.pkl ← Saved TF-IDF + Random Forest
■
■ ■ ■ ■ ml/ ← Machine Learning notebooks
■ ■ ■ ■ 01_eda.ipynb ← Exploratory Data Analysis
■ ■ ■ ■ 02_preprocessing.ipynb ← Data cleaning & ADASYN
■ ■ ■ ■ 03_baseline_model.ipynb ← TF-IDF + Random Forest
■ ■ ■ ■ 04_bert_model.ipynb ← BERT fine-tuning
■ ■ ■ ■ 05_shap_explainability.ipynb ← SHAP analysis
■ ■ ■ ■ utils.py ← Shared utility functions
■
■ ■ ■ ■ frontend/ ← React web application
■ ■ ■ ■ package.json ← Node dependencies
■ ■ ■ ■ tailwind.config.js ← TailwindCSS config
■ ■ ■ ■ public/
■ ■ ■ ■ index.html ← HTML entry point
■ ■ ■ ■ src/
■ ■ ■ ■ App.jsx ← Main app with routing
■ ■ ■ ■ index.jsx ← React DOM render
■ ■ ■ ■ api/
■ ■ ■ ■ analyzeApi.js ← All API call functions
■ ■ ■ ■ pages/
■ ■ ■ ■ HomePage.jsx ← Landing page
■ ■ ■ ■ AnalyzePage.jsx ← Main analysis page
■ ■ ■ ■ ResultPage.jsx ← Results display page
■ ■ ■ ■ components/
■ ■ ■ ■ TextInput.jsx ← Text paste component
■ ■ ■ ■ ImageUpload.jsx ← Image upload component
■ ■ ■ ■ UrlInput.jsx ← URL input component
```

- ■■■ RiskGauge.jsx ← Risk score gauge
- ■■■ ReasonCards.jsx ← SHAP reason cards
- ■■■ Navbar.jsx ← Navigation bar
-
- datasets/
 - ■■■ raw/ ← Original Kaggle CSV
 - ■■■ processed/ ← Cleaned + preprocessed CSV
 - ■■■ augmented/ ← ADASYN-balanced dataset
 -
- docs/ ← Project documentation
 - ■■■ SRS.docx ← Software Requirements Spec
 - ■■■ architecture_diagram.png ← System architecture image
 - ■■■ wireframes/ ← UI wireframes
 -
- .env ← Environment variables (DO NOT commit)
- .gitignore ← Files to exclude from Git
- README.md ← Project description

How to Create This Structure — Commands:

■ Create all folders and files

```
# Run these commands in your terminal (one by one)
mkdir fake-job-detector
cd fake-job-detector
mkdir backend backend/routers backend/services backend/models
mkdir ml
mkdir frontend
mkdir datasets datasets/raw datasets/processed datasets/augmented
mkdir docs docs/wireframes
touch README.md .gitignore .env
touch backend/main.py backend/config.py backend/requirements.txt
touch backend/routers/analyze.py backend/routers/health.py
touch backend/services/text_analyzer.py backend/services/image_analyzer.py
touch backend/services/url_checker.py backend/services/risk_scorer.py
```

Step 1.1 — Install Required Software

1	Python 3.10+ Download from python.org/downloads — Check 'Add Python to PATH' during install
2	Node.js 18+ Download from nodejs.org — This also installs npm automatically
3	Git Download from git-scm.com — For version control
4	VS Code Download from code.visualstudio.com — Best code editor
5	Postman Download from postman.com — For testing API endpoints

Step 1.2 — VS Code Extensions to Install

Open VS Code → Click Extensions (Ctrl+Shift+X) → Search and install these:

- **Python** (by Microsoft) — Python language support
- **Pylance** (by Microsoft) — Python type checking
- **ES7 React Snippets** — React code shortcuts
- **Tailwind CSS IntelliSense** — CSS class suggestions
- **Thunder Client** — API testing inside VS Code
- **GitLens** — Enhanced Git integration

Step 1.3 — Create Python Virtual Environment

A virtual environment isolates your project's Python packages from the rest of your computer. ALWAYS use a virtual environment. Never install packages globally.

■ Create & activate virtual environment

```
# Navigate to your project folder
cd fake-job-detector/backend

# Create the virtual environment (named 'venv')
python -m venv venv

# Activate it — Windows:
venv\Scripts\activate

# Activate it — Mac/Linux:
source venv/bin/activate

# You will see (venv) at the start of your terminal line — that means it is active
# Every time you open a new terminal, you MUST activate the venv again
```

Step 1.4 — Install ALL Python Packages

Create **backend/requirements.txt** with exactly this content:

■ requirements.txt — exact versions

```
# backend/requirements.txt
fastapi==0.104.1
uvicorn[standard]==0.24.0
python-multipart==0.0.6
pydantic==2.4.2
requests==2.31.0
python-whois==0.8.0
pandas==2.1.1
numpy==1.24.4
scikit-learn==1.3.2
imbalanced-learn==0.11.0
transformers==4.35.0
torch==2.0.1
Pillow==10.1.0
shap==0.43.0
joblib==1.3.2
python-dotenv==1.0.0
httpx==0.25.1
```

■ Install all packages

```
# Now install everything (make sure venv is active first)
pip install -r requirements.txt

# This will take 5-10 minutes – PyTorch is large
# If torch install fails, install it separately:
pip install torch --index-url https://download.pytorch.org/whl/cpu
```

Step 1.5 — Setup Git Repository

■ Setup Git & GitHub

```
# Go to root project folder
cd fake-job-detector

# Initialize git
git init

# Create .gitignore file
echo 'venv/' >> .gitignore
echo '__pycache__/' >> .gitignore
echo '.env' >> .gitignore
echo '*.pkl' >> .gitignore
echo 'node_modules/' >> .gitignore
echo 'datasets/raw/' >> .gitignore

# Go to GitHub, create a new repository named 'fake-job-detector'
# Then connect your local repo to GitHub:
git remote add origin https://github.com/YOUR_USERNAME/fake-job-detector.git
git add .
git commit -m "Initial project structure"
git push -u origin main
```

Step 2.1 — Download the Dataset

We use the **EMSCAD dataset** (Employment Scam Aegean Dataset) from Kaggle. This is the standard benchmark dataset used in all 4 of your research papers.

1	Go to Kaggle Open browser → Go to kaggle.com/shivamb/real-or-fake-fake-jobposting-prediction
2	Create account Sign up for free on Kaggle if you don't have an account
3	Download Click the Download button → You get 'fake_job_postings.csv'
4	Place file Copy the CSV file to: <code>fake-job-detector/datasets/raw/fake_job_postings.csv</code>
5	Verify Open the file — it should have 17,880 rows and 18 columns including 'fraudulent'

Step 2.2 — Dataset Columns Explained

Column Name	Data Type	Description	Important?
job_id	int	Unique ID for each job post	No — drop this
title	text	Job title (e.g. 'Software Engineer')	YES — key feature
location	text	City, State, Country	YES — scams often vague
department	text	Department name	Moderate
salary_range	text	Salary (often blank in fakes)	YES — fakes are vague
company_profile	text	About the company	YES — fakes have none
description	text	Full job description	YES — most important
requirements	text	Skills required	YES
benefits	text	Benefits offered	Moderate
telecommuting	int (0/1)	Work from home option	YES — fakes say yes
has_company_logo	int (0/1)	Company logo present?	YES — fakes often no
has_questions	int (0/1)	Application has screening questions?	YES
employment_type	text	Full-time, Part-time etc.	Moderate
required_experience	text	Experience level needed	Moderate
required_education	text	Education level needed	Moderate
industry	text	Industry sector	YES — fakes in accounting

function	text	Job function	Moderate
fraudulent	int (0/1)	TARGET — 0=real, 1=fake	YES — this is the label

Step 2.3 — Exploratory Data Analysis (EDA) — ml/01_eda.ipynb

Create a Jupyter notebook and run this code to understand your data:

■ EDA code — run in Jupyter notebook

```
# ml/01_eda.ipynb — Run each cell one by one
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load the dataset
df = pd.read_csv('../datasets/raw/fake_job_postings.csv')
print('Shape:', df.shape) # Should be (17880, 18)
print(df.head())
print(df.info())

# 2. Check class distribution (very important!)
print(df['fraudulent'].value_counts())
# Output: 0 (real) = 17014, 1 (fake) = 866
# Only 4.84% are fake — this is class imbalance!

# 3. Visualize class distribution
df['fraudulent'].value_counts().plot(kind='bar', color=['green', 'red'])
plt.title('Real vs Fake Job Posts')
plt.xlabel('0=Real, 1=Fake')
plt.savefig('../docs/class_distribution.png')
plt.show()

# 4. Check missing values
print(df.isnull().sum())
# salary_range, department, company_profile, description etc. have NaN

# 5. Fake job patterns — what sectors?
fake_jobs = df[df['fraudulent'] == 1]
print('Top industries in fake jobs:')
print(fake_jobs['industry'].value_counts().head(10))

# 6. Correlation — has_company_logo vs fraudulent
print(pd.crosstab(df['has_company_logo'], df['fraudulent']))
# Result: Jobs WITHOUT logo are much more likely fake

# 7. Word frequency analysis — what words appear in fake posts?
from collections import Counter
import re
fake_text = ' '.join(df[df['fraudulent']==1]['description'].dropna())
real_text = ' '.join(df[df['fraudulent']==0]['description'].dropna())
fake_words = Counter(fake_text.lower().split()).most_common(20)
print('Most common words in FAKE posts:', fake_words)
```

Step 2.4 — Data Cleaning & Preprocessing — ml/02_preprocessing.ipynb

■ Data cleaning & preprocessing

```

# ml/02_preprocessing.ipynb
import pandas as pd
import numpy as np
import re
from sklearn.model_selection import train_test_split

df = pd.read_csv('../datasets/raw/fake_job_postings.csv')

# STEP 1: Fill missing values with empty string
text_cols = ['title', 'location', 'department', 'company_profile',
'description', 'requirements', 'benefits', 'industry', 'function']
df[text_cols] = df[text_cols].fillna('')

# STEP 2: Combine all text columns into one 'combined_text' feature
# This is the most important step – gives the model maximum context
df['combined_text'] = (df['title'] + ' ' + df['company_profile'] + ' ' +
df['description'] + ' ' + df['requirements'] + ' ' +
df['benefits'] + ' ' + df['industry'])

# STEP 3: Clean the text
def clean_text(text):
text = str(text).lower() # lowercase everything
text = re.sub(r'<[^>]+>', ' ', text) # remove HTML tags
text = re.sub(r'http\S+', ' ', text) # remove URLs
text = re.sub(r'[^a-z0-9\s]', ' ', text) # keep only alphanumeric
text = re.sub(r'\s+', ' ', text).strip() # remove extra spaces
return text

df['cleaned_text'] = df['combined_text'].apply(clean_text)

# STEP 4: Handle class imbalance using ADASYN
# ADASYN creates synthetic samples of the minority class (fake posts)
from imblearn.over_sampling import ADASYN
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000)
X = tfidf.fit_transform(df['cleaned_text']).toarray()
y = df['fraudulent'].values

adasyn = ADASYN(random_state=42)
X_res, y_res = adasyn.fit_resample(X, y)
print('Before ADASYN:', dict(zip(*np.unique(y, return_counts=True))))
print('After ADASYN:', dict(zip(*np.unique(y_res, return_counts=True))))
# After: fake posts will be ~15% of total

# STEP 5: Train/Test split – 80% train, 20% test
X_train, X_test, y_train, y_test = train_test_split(
X_res, y_res, test_size=0.2, random_state=42, stratify=y_res)

# STEP 6: Save processed data
df[['cleaned_text', 'fraudulent']].to_csv('../datasets/processed/clean_data.csv',
index=False)
print('Saved processed dataset!')

```

Step 3.1 — Baseline Model: TF-IDF + Random Forest — ml/03_baseline_model.ipynb

Start with the baseline model first. This is fast to train and gives ~98% accuracy. Use it to verify your data pipeline works before moving to BERT.

■ TF-IDF + Random Forest — baseline model

```
# ml/03_baseline_model.ipynb
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import joblib
import shap
import matplotlib.pyplot as plt

# Load preprocessed data
df = pd.read_csv('../datasets/processed/clean_data.csv')
X_text = df['cleaned_text']
y = df['fraudulent']

# TF-IDF vectorization
# max_features=10000: keep top 10000 most important words
# ngram_range=(1,2): use single words AND pairs of words
tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X = tfidf.fit_transform(X_text)

# Train/Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Train Random Forest
# n_estimators=500: use 500 trees (more = better but slower)
# n_jobs=-1: use all CPU cores to speed up training
# class_weight='balanced': handles remaining imbalance automatically
rf = RandomForestClassifier(n_estimators=500, random_state=42,
    n_jobs=-1, class_weight='balanced')
rf.fit(X_train, y_train)

# Evaluate the model
y_pred = rf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=['Real', 'Fake']))
# Expected output: ~98% accuracy

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print('Confusion Matrix:')
print(cm)
# Row 0: Real posts (TN, FP)
# Row 1: Fake posts (FN, TP) — minimize FN (fake missed as real)
```

```
# Save model and vectorizer – we will load these in FastAPI
import os
os.makedirs('../backend/models', exist_ok=True)
joblib.dump(rf, '../backend/models/rf_model.pkl')
joblib.dump(tfidf, '../backend/models/tfidf_vectorizer.pkl')
print('Models saved!')
```

Step 3.2 — Advanced Model: Fine-tune BERT — ml/04_bert_model.ipynb

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model that understands context much better than TF-IDF. Fine-tuning means taking the pre-trained BERT and training it further on our specific dataset. This takes longer but achieves higher accuracy (99%+).

■ BERT fine-tuning — advanced model

```
# ml/04_bert_model.ipynb
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import AdamW, get_linear_schedule_with_warmup
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import pandas as pd
import numpy as np

# Check if GPU is available (training is MUCH faster with GPU)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')
# If CPU – training will take ~2 hours. GPU – ~20 minutes

# Load data
df = pd.read_csv('../datasets/processed/clean_data.csv')
texts = df['cleaned_text'].tolist()
labels = df['fraudulent'].tolist()

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42, stratify=labels)

# Load BERT tokenizer
# 'bert-base-uncased' = pre-trained on 3.3 billion words
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# Create PyTorch Dataset class
class JobDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=256):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            max_length=self.max_len,
```

```

padding='max_length',
truncation=True,
return_tensors='pt'
)
return {
'input_ids': encoding['input_ids'].squeeze(),
'attention_mask': encoding['attention_mask'].squeeze(),
'label': torch.tensor(self.labels[idx], dtype=torch.long)
}

# Create DataLoaders
train_dataset = JobDataset(X_train, y_train, tokenizer)
test_dataset = JobDataset(X_test, y_test, tokenizer)
train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=16)

# Load BERT model for binary classification (num_labels=2)
model = BertForSequenceClassification.from_pretrained(
'bert-base-uncased', num_labels=2)
model = model.to(device)

# Optimizer and learning rate scheduler
optimizer = AdamW(model.parameters(), lr=2e-5, weight_decay=0.01)
total_steps = len(train_loader) * 3 # 3 epochs
scheduler = get_linear_schedule_with_warmup(
optimizer, num_warmup_steps=0, num_training_steps=total_steps)

# Training loop – 3 epochs
EPOCHS = 3
for epoch in range(EPOCHS):
model.train()
total_loss = 0
for batch in train_loader:
optimizer.zero_grad()
input_ids = batch['input_ids'].to(device)
attention_mask = batch['attention_mask'].to(device)
labels_batch = batch['label'].to(device)
outputs = model(input_ids=input_ids,
attention_mask=attention_mask,
labels=labels_batch)
loss = outputs.loss
total_loss += loss.item()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
scheduler.step()
avg_loss = total_loss / len(train_loader)
print(f'Epoch {epoch+1}/{EPOCHS} – Loss: {avg_loss:.4f}')

# Evaluate
model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
for batch in test_loader:
input_ids = batch['input_ids'].to(device)
attention_mask = batch['attention_mask'].to(device)
outputs = model(input_ids=input_ids, attention_mask=attention_mask)
preds = torch.argmax(outputs.logits, dim=1).cpu().numpy()
all_preds.extend(preds)

```

```

all_labels.extend(batch['label'].numpy())

print(classification_report(all_labels, all_preds, target_names=['Real', 'Fake']))

# Save the fine-tuned BERT model
model.save_pretrained('../backend/models/bert_model')
tokenizer.save_pretrained('../backend/models/bert_model')
print('BERT model saved!')

```

Step 3.3 — SHAP Explainability — ml/05_shap_explainability.ipynb

SHAP (SHapley Additive exPlanations) tells us WHICH words/features caused the model to classify a post as fake. This is what powers the 'Reason Cards' on the frontend. We apply SHAP to the Random Forest model (faster than BERT for inference).

■ SHAP explainability

```

# ml/05_shap_explainability.ipynb
import shap
import joblib
import numpy as np

# Load saved model and vectorizer
rf = joblib.load('../backend/models/rf_model.pkl')
tfidf = joblib.load('../backend/models/tfidf_vectorizer.pkl')

# Create SHAP explainer
explainer = shap.TreeExplainer(rf)

# Test on a sample fake post
sample_text = 'Work from home earn 50000 per month no experience needed apply now cash daily'
sample_vec = tfidf.transform([sample_text]).toarray()
shap_values = explainer.shap_values(sample_vec)

# Get top contributing features (words)
feature_names = tfidf.get_feature_names_out()
shap_for_fake = shap_values[1][0] # SHAP values for class 1 (fake)
top_indices = np.argsort(np.abs(shap_for_fake))[:-1][:10]
top_words = [(feature_names[i], shap_for_fake[i]) for i in top_indices]
print('Top words causing fake classification:')
for word, val in top_words:
    print(f' {word}: {val:.4f}')
# This output becomes the Reason Cards on frontend

# Save explainer
joblib.dump(explainer, '../backend/models/shap_explainer.pkl')
print('SHAP explainer saved!')

```

backend/services/url_checker.py — Complete Code

This module checks if a URL in the job ad is safe. It uses 3 checks: Google Safe Browsing API (detects phishing), domain age (new domains are suspicious), and redirect chain analysis (too many redirects = suspicious).

■ url_checker.py — complete code

```
# backend/services/url_checker.py
import requests
import whois
from datetime import datetime, timezone
import os

GOOGLE_SAFE_BROWSING_KEY = os.getenv('GOOGLE_SAFE_BROWSING_KEY', '')

def check_url_safety(url: str) -> dict:
    """
    Returns a dict with:
    - is_safe: bool
    - risk_score: int (0-100)
    - flags: list of strings explaining what was found
    """
    flags = []
    risk = 0

    # 1. Check with Google Safe Browsing API
    if GOOGLE_SAFE_BROWSING_KEY:
        try:
            api_url =
            f'https://safebrowsing.googleapis.com/v4/threatMatches:find?key={GOOGLE_SAFE_BROWSING_KEY}'
            payload = {
                'client': {'clientId': 'fake-job-detector', 'clientVersion': '1.0'},
                'threatInfo': {
                    'threatTypes': ['MALWARE', 'SOCIAL_ENGINEERING', 'UNWANTED_SOFTWARE'],
                    'platformTypes': ['ANY_PLATFORM'],
                    'threatEntryTypes': ['URL'],
                    'threatEntries': [{'url': url}]
                }
            }
            resp = requests.post(api_url, json=payload, timeout=5)
            data = resp.json()
            if data.get('matches'):
                flags.append('URL flagged by Google Safe Browsing as phishing/malware')
                risk += 60
        except Exception as e:
            flags.append(f'Could not verify URL with Safe Browsing: {str(e)}')

    # 2. Check domain age using WHOIS
    try:
        from urllib.parse import urlparse
        domain = urlparse(url).netloc.replace('www.', '')
        w = whois.whois(domain)
        creation_date = w.creation_date
        if isinstance(creation_date, list):

```

```

creation_date = creation_date[0]
if creation_date:
    if creation_date.tzinfo is None:
        creation_date = creation_date.replace(tzinfo=timezone.utc)
    now = datetime.now(timezone.utc)
    age_days = (now - creation_date).days
    if age_days < 30:
        flags.append(f'Domain created only {age_days} days ago – very new, suspicious')
        risk += 40
    elif age_days < 90:
        flags.append(f'Domain is relatively new ({age_days} days old)')
        risk += 20
    except Exception:
        flags.append('Could not determine domain age')
        risk += 10

# 3. Check for redirect chains
try:
    resp = requests.get(url, allow_redirects=True, timeout=5)
    if len(resp.history) > 3:
        flags.append(f'URL has {len(resp.history)} redirects – suspicious')
        risk += 20
    # Check if final URL domain differs from original
    from urllib.parse import urlparse
    orig_domain = urlparse(url).netloc
    final_domain = urlparse(resp.url).netloc
    if orig_domain != final_domain:
        flags.append(f'URL redirects to different domain: {final_domain}')
        risk += 25
    except Exception:
        flags.append('Could not follow URL redirects')
        risk += 5

risk = min(risk, 100) # cap at 100
return {
    'is_safe': risk < 40,
    'risk_score': risk,
    'flags': flags
}

```

How to get Google Safe Browsing API Key (Free):

1	Go to Google Cloud Console Open console.cloud.google.com in your browser
2	Create Project Click 'New Project' → Name it 'fake-job-detector' → Click Create
3	Enable API Go to APIs & Services → Enable APIs → Search 'Safe Browsing' → Enable it
4	Create Key Go to APIs & Services → Credentials → Create API Key → Copy the key
5	Add to .env Open your .env file → Add: GOOGLE_SAFE_BROWSING_KEY=your_key_here

backend/services/image_analyzer.py — Complete Code

This module uses OpenAI's CLIP model to analyze job ad screenshots. CLIP understands both images and text together. We use it to detect: stock photos pretending to be offices, suspicious text in images, and check for known company logo misuse.

■ image_analyzer.py — complete code

```
# backend/services/image_analyzer.py
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch
import io

# Load CLIP model (loads once at startup)
clip_model = CLIPModel.from_pretrained('openai/clip-vit-base-patch32')
clip_processor = CLIPProcessor.from_pretrained('openai/clip-vit-base-patch32')
clip_model.eval()

# These are the text prompts we compare the image against
# CLIP measures how similar the image is to each text description
SUSPICIOUS_PROMPTS = [
    'a fake job advertisement with unrealistic salary',
    'a scam job posting with work from home',
    'a phishing advertisement asking for personal information',
    'a stock photo office that looks fake',
    'a WhatsApp message with job offer',
]

LEGIT_PROMPTS = [
    'a professional company job advertisement',
    'a legitimate corporate hiring poster',
    'an official company recruitment banner',
]

def analyze_image(image_bytes: bytes) -> dict:
    """
    Takes image as bytes, returns analysis dict with:
    - risk_score: int (0-100)
    - flags: list of suspicious findings
    - confidence: float
    """
    try:
        image = Image.open(io.BytesIO(image_bytes)).convert('RGB')
        flags = []
        risk = 0

        all_prompts = SUSPICIOUS_PROMPTS + LEGIT_PROMPTS
        inputs = clip_processor(
            text=all_prompts,
            images=image,
            return_tensors='pt',
            padding=True
        )

        with torch.no_grad():
```

```

outputs = clip_model(**inputs)
probs = outputs.logits_per_image.softmax(dim=1)[0].tolist()

# Scores for suspicious vs legit prompts
suspicious_scores = probs[:len(SUSPICIOUS_PROMPTS)]
legit_scores = probs[len(SUSPICIOUS_PROMPTS):]

max_suspicious = max(suspicious_scores)
max_legit = max(legit_scores)

if max_suspicious > 0.3:
    susp_idx = suspicious_scores.index(max_suspicious)
    flags.append(f'Image resembles: {SUSPICIOUS_PROMPTS[susp_idx]}')
    risk += int(max_suspicious * 80)

if max_legit > max_suspicious:
    flags.append('Image appears to be from a professional company')
    risk = max(0, risk - 20)

# Check image dimensions – very small images are suspicious (copied from web)
w, h = image.size
if w < 200 or h < 200:
    flags.append(f'Very small image ({w}x{h}px) – may be a copied thumbnail')
    risk += 15

risk = min(risk, 100)
return {
    'risk_score': risk,
    'flags': flags,
    'confidence': round(max_suspicious, 3)
}
except Exception as e:
    return {'risk_score': 0, 'flags': [f'Image analysis failed: {str(e)}'], 'confidence': 0}

```

backend/services/text_analyzer.py — Load & Use ML Models

■ text_analyzer.py — load models & predict

```
# backend/services/text_analyzer.py
import joblib
import re
import numpy as np
from transformers import BertTokenizer, BertForSequenceClassification
import torch
import os

# Load both models at startup — only loaded once
BASE = os.path.dirname(os.path.dirname(__file__))
rf_model = joblib.load(os.path.join(BASE, 'models', 'rf_model.pkl'))
tfidf = joblib.load(os.path.join(BASE, 'models', 'tfidf_vectorizer.pkl'))
explainer = joblib.load(os.path.join(BASE, 'models', 'shap_explainer.pkl'))

# Load BERT model (optional — use if available)
BERT_PATH = os.path.join(BASE, 'models', 'bert_model')
USE_BERT = os.path.exists(BERT_PATH)
if USE_BERT:
    bert_tokenizer = BertTokenizer.from_pretrained(BERT_PATH)
    bert_model = BertForSequenceClassification.from_pretrained(BERT_PATH)
    bert_model.eval()

def clean_text(text: str) -> str:
    text = str(text).lower()
    text = re.sub(r'<[^>]+>', ' ', text)
    text = re.sub(r'http\S+', '', text)
    text = re.sub(r'[\^a-z0-9\s]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

def analyze_text(text: str) -> dict:
    """Returns risk_score, prediction, probability, and top_reasons (SHAP)"""
    cleaned = clean_text(text)

    # Random Forest prediction
    vec = tfidf.transform([cleaned]).toarray()
    rf_prob = rf_model.predict_proba(vec)[0][1] # prob of being fake
    rf_pred = 1 if rf_prob > 0.5 else 0

    # BERT prediction (if model exists)
    bert_prob = rf_prob # default to RF
    if USE_BERT:
        inputs = bert_tokenizer(cleaned, return_tensors='pt',
                                max_length=256, truncation=True, padding=True)
        with torch.no_grad():
            logits = bert_model(**inputs).logits
            bert_prob = torch.softmax(logits, dim=1)[0][1].item()

    # Ensemble: weighted average (BERT gets more weight)
    final_prob = 0.4 * rf_prob + 0.6 * bert_prob if USE_BERT else rf_prob
```

```

final_pred = 1 if final_prob > 0.5 else 0

# SHAP: get top 5 reasons
shap_vals = explainer.shap_values(vec)[1][0]
feat_names = tfidf.get_feature_names_out()
top_idx = np.argsort(np.abs(shap_vals))[:, -1][:5]
reasons = [{'word': feat_names[i], 'impact': round(float(shap_vals[i]), 4)}
            for i in top_idx]

return {
    'prediction': final_pred,
    'probability': round(float(final_prob), 3),
    'risk_score': int(final_prob * 100),
    'top_reasons': reasons
}

```

backend/services/risk_scorer.py — Combine All Signals

■ risk_scorer.py — combine all signals

```

# backend/services/risk_scorer.py

def calculate_final_risk(text_result=None, image_result=None, url_result=None) -> dict:
    """
    Combines text, image, and URL risk scores into one final risk score.
    Weights: text=60%, image=25%, url=15%
    """
    weighted_score = 0
    all_flags = []
    components = {}

    if text_result:
        score = text_result.get('risk_score', 0)
        weighted_score += score * 0.60
        all_flags.extend([f"[TEXT] {r['word']}" (impact: {r['impact']})"
                          for r in text_result.get('top_reasons', [])])
        components['text'] = score

    if image_result:
        score = image_result.get('risk_score', 0)
        weighted_score += score * 0.25
        all_flags.extend([f"[IMAGE] {f}" for f in image_result.get('flags', [])])
        components['image'] = score

    if url_result:
        score = url_result.get('risk_score', 0)
        weighted_score += score * 0.15
        all_flags.extend([f"[URL] {f}" for f in url_result.get('flags', [])])
        components['url'] = score

    final_score = min(int(weighted_score), 100)

    if final_score >= 70:
        label = 'LIKELY SCAM'
        color = 'red'
    elif final_score >= 40:
        label = 'SUSPICIOUS'
        color = 'orange'
    else:

```

```

label = 'LIKELY SAFE'
color = 'green'

return {
    'final_score': final_score,
    'label': label,
    'color': color,
    'flags': all_flags,
    'components': components
}

```

backend/main.py — FastAPI Application

■ main.py — complete FastAPI app

```

# backend/main.py
from fastapi import FastAPI, UploadFile, File, Form
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import Optional
from services.text_analyzer import analyze_text
from services.image_analyzer import analyze_image
from services.url_checker import check_url_safety
from services.risk_scorer import calculate_final_risk

app = FastAPI(
    title='Fake Job Detector API',
    description='Detects fake job postings using NLP, Vision, and URL analysis',
    version='1.0.0'
)

# CORS — allows React frontend to call this API
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'], # In production: replace with your Vercel URL
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

class TextRequest(BaseModel):
    text: str

class URLRequest(BaseModel):
    url: str

# ENDPOINT 1: Health check
@app.get('/health')
def health():
    return {'status': 'ok', 'message': 'Fake Job Detector API is running'}

# ENDPOINT 2: Analyze text
@app.post('/analyze/text')
def analyze_text_endpoint(req: TextRequest):
    text_result = analyze_text(req.text)
    final = calculate_final_risk(text_result=text_result)
    return {'text_analysis': text_result, 'final_result': final}

# ENDPOINT 3: Analyze image

```

```

@app.post('/analyze/image')
async def analyze_image_endpoint(file: UploadFile = File(...)):
    image_bytes = await file.read()
    image_result = analyze_image(image_bytes)
    final = calculate_final_risk(image_result=image_result)
    return {'image_analysis': image_result, 'final_result': final}

# ENDPOINT 4: Analyze URL
@app.post('/analyze/url')
def analyze_url_endpoint(req: URLRequest):
    url_result = check_url_safety(req.url)
    final = calculate_final_risk(url_result=url_result)
    return {'url_analysis': url_result, 'final_result': final}

# ENDPOINT 5: Analyze everything at once (full analysis)
@app.post('/analyze/full')
async def analyze_full(
    text: Optional[str] = Form(None),
    url: Optional[str] = Form(None),
    file: Optional[UploadFile] = File(None)
):
    text_result = analyze_text(text) if text else None
    image_result = analyze_image(await file.read()) if file else None
    url_result = check_url_safety(url) if url else None
    final = calculate_final_risk(text_result, image_result, url_result)
    return {
        'text_analysis': text_result,
        'image_analysis': image_result,
        'url_analysis': url_result,
        'final_result': final
    }

# Run with: uvicorn main:app --reload --port 8000

```

■ How to run the backend

```

# How to run the backend
cd fake-job-detector/backend
source venv/bin/activate # or venv\Scripts\activate on Windows
uvicorn main:app --reload --port 8000

# Open browser: http://localhost:8000/docs
# You will see the Swagger UI – test all endpoints here
# Try /analyze/text with: {'text': 'work from home earn 50000 daily'}

```

Step 7.1 — Setup React Project

■ Create React project

```
# Navigate to project root
cd fake-job-detector

# Create React app using Vite (faster than create-react-app)
npm create vite@latest frontend -- --template react
cd frontend

# Install all dependencies
npm install
npm install axios react-router-dom
npm install -D tailwindcss postcss autoprefixer

# Initialize Tailwind CSS
npx tailwindcss init -p

# This creates tailwind.config.js – update it:
# content: ['./index.html', './src/**/*.js,jsx']
```

■ src/index.css — add Tailwind directives

```
/* Add to src/index.css – top of the file */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

src/api/analyzeApi.js — All API Calls

■ analyzeApi.js — all API functions

```
// src/api/analyzeApi.js
import axios from 'axios';

const BASE_URL = process.env.REACT_APP_API_URL || 'http://localhost:8000';
const api = axios.create({ baseURL: BASE_URL });

// Analyze text only
export const analyzeText = async (text) => {
  const res = await api.post('/analyze/text', { text });
  return res.data;
};

// Analyze image only
export const analyzeImage = async (imageFile) => {
  const formData = new FormData();
  formData.append('file', imageFile);
  const res = await api.post('/analyze/image', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  });
  return res.data;
};
```

```
// Analyze URL only
export const analyzeUrl = async (url) => {
  const res = await api.post('/analyze/url', { url });
  return res.data;
};

// Full analysis (all at once)
export const analyzeAll = async ({ text, url, imageFile }) => {
  const formData = new FormData();
  if (text) formData.append('text', text);
  if (url) formData.append('url', url);
  if (imageFile) formData.append('file', imageFile);
  const res = await api.post('/analyze/full', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  });
  return res.data;
};
```

src/App.jsx — Main App with Routing

■ App.jsx

```
// src/App.jsx
import { BrowserRouter as Router, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import HomePage from './pages/HomePage';
import AnalyzePage from './pages/AnalyzePage';

function App() {
  return (
    <Router>
    <div className='min-h-screen bg-gray-50'>
    <Navbar />
    <Routes>
    <Route path='/' element={ <HomePage /> } />
    <Route path='/analyze' element={ <AnalyzePage /> } />
    </Routes>
    </div>
    </Router>
  );
}

export default App;
```

src/pages/AnalyzePage.jsx — Main Analysis Page (Full Code)

■ AnalyzePage.jsx — complete main page

```
// src/pages/AnalyzePage.jsx
import { useState } from 'react';
import { analyzeAll } from '../api/analyzeApi';
import RiskGauge from '../components/RiskGauge';
import ReasonCards from '../components/ReasonCards';

export default function AnalyzePage() {
  const [text, setText] = useState('');
  const [url, setUrl] = useState('');
  const [imageFile, setImageFile] = useState(null);
  const [result, setResult] = useState(null);
```

```

const [loading, setLoading] = useState(false);
const [error, setError] = useState('');

const handleSubmit = async (e) => {
  e.preventDefault();
  if (!text && !url && !imageFile) {
    setError('Please provide at least one input: text, URL, or image');
    return;
  }
  setLoading(true);
  setError('');
  try {
    const data = await analyzeAll({ text, url, imageFile });
    setResult(data);
  } catch (err) {
    setError('Analysis failed. Make sure the backend is running.');
```

} finally {
 setLoading(false);
 }
 }
};

return (
 <div className='max-w-4xl mx-auto p-6'>
 <h1 className='text-3xl font-bold text-blue-900 mb-2'>Analyze a Job Posting</h1>
 <p className='text-gray-600 mb-6'>
 Paste the job text, enter a URL, or upload an image – or all three.
 </p>

 <form onSubmit={handleSubmit} className='space-y-6'>
 <div>
 <div>
 <label className='block text-sm font-bold text-gray-700 mb-1'>
 Job Post Text
 </label>
 <textarea
 className='w-full border rounded-lg p-3 h-40 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500'
 placeholder='Paste the full job posting text here...'
 value={text}
 onChange={e => setText(e.target.value)}
 </>
 </div>

 <div>
 <label className='block text-sm font-bold text-gray-700 mb-1'>
 Job/Ad URL (optional)
 </label>
 <input
 type='url'
 className='w-full border rounded-lg p-3 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500'
 placeholder='https://www.example.com/jobs/12345'
 value={url}
 onChange={e => setUrl(e.target.value)}
 </>
 </div>

 <div>
 <label className='block text-sm font-bold text-gray-700 mb-1'>
 Image Upload
 </label>
 <input
 type='file'
 className='w-full border rounded-lg p-3 text-sm focus:outline-none focus:ring-2 focus:ring-blue-500'
 value={imageFile}
 onChange={e => setImageFile(e.target.files[0])}
 </>
 </div>
 </div>
 </form>
 </div>
);

```

<div>
<label className='block text-sm font-bold text-gray-700 mb-1'>
Ad Screenshot (optional)
</label>
<input
type='file'
accept='image/*'
className='w-full border rounded-lg p-3 text-sm'
onChange={e => setImageFile(e.target.files[0])}
/>
</div>

{error && <p className='text-red-600 text-sm'>{error}</p>}

<button
type='submit'
disabled={loading}
className='w-full bg-blue-800 text-white py-3 rounded-lg font-bold text-lg
hover:bg-blue-900 disabled:opacity-50 transition'
>
{loading ? 'Analyzing...' : 'Analyze Job Posting'}
</button>
</form>

{/* Results Section */}
{result && (
<div className='mt-10 space-y-6'>
<RiskGauge score={result.final_result.final_score}
label={result.final_result.label}
color={result.final_result.color} />
<ReasonCards flags={result.final_result.flags}
textReasons={result.text_analysis?.top_reasons} />
</div>
)}
</div>
);
}

```

src/components/RiskGauge.jsx — Risk Score Display

■ RiskGauge.jsx

```

// src/components/RiskGauge.jsx
export default function RiskGauge({ score, label, color }) {
const colorMap = {
green: { bg: 'bg-green-100', text: 'text-green-800', bar: 'bg-green-500' },
orange: { bg: 'bg-orange-100', text: 'text-orange-800', bar: 'bg-orange-500' },
red: { bg: 'bg-red-100', text: 'text-red-800', bar: 'bg-red-600' },
};
const c = colorMap[color] || colorMap.green;

return (
<div className={`rounded-2xl p-6 ${c.bg} border-2`}>
<div className='flex justify-between items-center mb-3'>
<h2 className='text-xl font-bold text-gray-800'>Risk Assessment</h2>
<span className={`text-2xl font-bold ${c.text}`}>{label}</span>
</div>
<div className='flex items-center gap-4'>
<span className={`text-5xl font-black ${c.text}`}>{score}</span>

```

```

<div className='flex-1'>
<div className='w-full bg-gray-200 rounded-full h-4'>
<div
className={`h-4 rounded-full transition-all duration-700 ${c.bar}`}
style={{ width: `${score}%` }}
/>
</div>
<p className='text-xs text-gray-500 mt-1'>Risk Score (0 = Safe, 100 = Definite Scam)</p>
</div>
</div>
</div>
);
}

```

src/components/ReasonCards.jsx — Show Why It Was Flagged

■ ReasonCards.jsx

```

// src/components/ReasonCards.jsx
export default function ReasonCards({ flags, textReasons }) {
  return (
    <div>
      <h3 className='text-lg font-bold text-gray-800 mb-3'>Why This Was Flagged</h3>

      {/* SHAP Word Reasons */}
      {textReasons && textReasons.length > 0 && (
        <div className='mb-4'>
          <p className='text-sm font-semibold text-gray-600 mb-2'>
            Suspicious words detected in text:
          </p>
          <div className='flex flex-wrap gap-2'>
            {textReasons.map((r, i) => (
              <span key={i}
                className={`px-3 py-1 rounded-full text-sm font-medium ${
                  r.impact > 0 ? 'bg-red-100 text-red-800' : 'bg-green-100 text-green-800'
                }`}>
                {r.word} {r.impact > 0 ? '+' : ''}{r.impact}
              </span>
            ))}
          </div>
        </div>
      )}

      {/* All Flags */}
      {flags && flags.length > 0 && (
        <div className='space-y-2'>
          {flags.map((flag, i) => (
            <div key={i} className='flex items-start gap-2 bg-yellow-50 border border-yellow-200
              rounded-lg p-3'>
              <span className='text-yellow-600 text-lg'>■■■</span>
              <p className='text-sm text-gray-800'>{flag}</p>
            </div>
          ))}
        </div>
      )}

      {(!flags || flags.length === 0) && (
        <p className='text-green-700 font-medium'>No suspicious signals detected.</p>
      )}
    </div>
  )
}

```

```
</div>  
);  
}
```

■ How to run the frontend

```
# How to run the frontend  
cd fake-job-detector/frontend  
npm run dev  
  
# Open browser: http://localhost:5173  
# Make sure backend is ALSO running at http://localhost:8000
```

Step 8.1 — Integration Checklist

- Backend is running on `http://localhost:8000`
- Frontend is running on `http://localhost:5173`
- Frontend can call `/analyze/text` — no CORS errors in browser console
- Frontend can call `/analyze/image` — file upload works
- Frontend can call `/analyze/url` — URL results show on screen
- RiskGauge displays the correct score and color
- ReasonCards show the SHAP word highlights
- Error message shows if backend is offline

Step 8.2 — Test Cases — Run Every One

Test ID	Input	Expected Result	Pass/Fail
TC-01	Paste known scam text: 'Work from home earn \$10000/month, no experience needed WhatsApp now'	Risk Score > 70, Label: LIKELY SCAM	
TC-02	Paste real Infosys job description (copy from Naukri)	Risk Score < 40, Label: LIKELY SAFE	
TC-03	URL: <code>https://www.linkedin.com/jobs</code>	Risk Score < 30, URL safe	
TC-04	URL of a known phishing site (use PhishTank.com for sample)	Risk Score > 50, URL flagged	
TC-05	Upload a real company job ad screenshot (e.g. from Naukri)	Risk Score < 40, no suspicious image flags	
TC-06	Upload a fake WhatsApp screenshot with job offer	Risk Score > 50, image flagged as suspicious	
TC-07	Submit form with NO input at all	Error message: 'Please provide at least one input'	
TC-08	Submit with invalid URL (not <code>http://...</code>)	URL analysis fails gracefully, still returns text result	
TC-09	Submit very long text (10000 words)	API handles it — no timeout, result returns within 10 seconds	
TC-10	Check <code>/health</code> endpoint in Postman	Returns: <code>{'status': 'ok', ...}</code>	

Step 9.1 — Deploy Backend on Render (Free)

1	Go to Render Open render.com → Sign up with your GitHub account
2	Create Web Service Click 'New' → 'Web Service' → Connect your GitHub repo
3	Select folder Root Directory: backend — Build Command: <code>pip install -r requirements.txt</code>
4	Start command Start Command: <code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code>
5	Add env vars In Render dashboard: Environment → Add <code>GOOGLE_SAFE_BROWSING_KEY</code> = your key
6	Deploy Click Deploy. Wait 5 minutes. Your backend URL will be: https://your-app.onrender.com
7	Verify Open https://your-app.onrender.com/health → Should return <code>{'status': 'ok'}</code>

Step 9.2 — Deploy Frontend on Vercel (Free)

1	Go to Vercel Open vercel.com → Sign up with your GitHub account
2	Import project Click 'New Project' → Import your GitHub repo → Select 'frontend' folder
3	Add env var Environment Variables: <code>REACT_APP_API_URL</code> = https://your-app.onrender.com
4	Deploy Click Deploy. Wait 2 minutes. Your app will be live at: https://your-app.vercel.app
5	Test live Open the Vercel URL → Test all 3 input types → Verify everything works
6	Share URL This is your live deployed URL — submit this for the project demo

Step 10.1 — README.md Structure

- Project Title & Description (1 paragraph)
- Live Demo Link (Vercel URL)
- Screenshot of the working app
- Tech Stack list
- Features list (text + image + URL + SHAP)
- How to Run Locally — step by step for backend and frontend
- API Documentation — list all endpoints with example requests
- Dataset — link to Kaggle EMSCAD
- Model Accuracy — table showing RF vs BERT results
- Team Members & Roles
- References — the 4 research papers

Step 10.2 — Presentation (PPT) Structure — 10 Slides

Slide	Title	Content
1	Title Slide	Project name, team names, college, date
2	Problem Statement	Stats on fake jobs, who is affected, why it matters
3	Existing Solutions & Gaps	What others have done, what we improve on
4	Our Solution Overview	What the system does, 3 input types, output
5	System Architecture	The 5-layer architecture diagram from this document
6	Tech Stack	Python + React + FastAPI + BERT + CLIP — with logos
7	ML Model Results	Accuracy table: RF vs BERT, confusion matrix screenshot
8	Live Demo	Open live URL, paste a scam job, show result in real time
9	Future Scope	Browser extension, Indian language support, more platforms
10	Team & Contributions	Each member's role and what they built

15. TEAM TASK ASSIGNMENTS — ALL PHASES

P1 — ML Engineer (Text)

- 1 Download & explore EMSCAD dataset (EDA notebook)
- 2 Write data cleaning & preprocessing code
- 3 Implement ADASYN class balancing
- 4 Train TF-IDF + Random Forest baseline model
- 5 Fine-tune BERT classifier (3 epochs)
- 6 Implement SHAP explainability
- 7 Save all model files (.pkl and bert_model/)
- 8 Write backend/services/text_analyzer.py
- 9 Measure accuracy, precision, recall, F1 for both models
- 10 Write ML section of final report

P2 — ML Engineer (Image + URL)

- 1 Build backend/services/image_analyzer.py (CLIP model)
- 2 Test CLIP with real vs fake ad screenshots
- 3 Build backend/services/url_checker.py
- 4 Register & test Google Safe Browsing API
- 5 Test URL checker with PhishTank sample URLs
- 6 Build backend/services/risk_scorer.py
- 7 Write backend/services/meta_api.py (Meta Ad Library)
- 8 Test all service modules independently
- 9 Document service APIs
- 10 Collect 20 test images (real + fake ads)

P3 — Backend Developer

- 1 Setup FastAPI project structure
- 2 Write backend/main.py with all 5 endpoints
- 3 Setup CORS middleware
- 4 Write backend/config.py for env variables

5	Test all endpoints using Postman
6	Integrate text_analyzer, image_analyzer, url_checker, risk_scorer
7	Handle error cases gracefully (try/except everywhere)
8	Deploy backend to Render
9	Configure environment variables on Render
10	Write API documentation section of README

P4 — Frontend Developer

1	Create React + Vite project
2	Setup TailwindCSS
3	Build Navbar component
4	Build HomePage with project intro
5	Build AnalyzePage with 3 input sections
6	Build RiskGauge component
7	Build ReasonCards component
8	Write analyzeApi.js with all API functions
9	Test integration with backend (CORS, file upload)
10	Deploy frontend to Vercel with correct API URL env variable

16. COMMON ERRORS & HOW TO FIX THEM

■ ERROR: ModuleNotFoundError: No module named 'fastapi'

■ FIX: Virtual environment is not activated. Run: `source venv/bin/activate` (Mac/Linux) or `venv\Scripts\activate` (Windows), then `pip install -r requirements.txt`

■ ERROR: CORS error in browser console

■ FIX: FastAPI CORS is not configured. Make sure `main.py` has `CORSMiddleware` with `allow_origins=['*']`. Also make sure the backend is actually running.

■ ERROR: torch.cuda.OutOfMemoryError during BERT training

■ FIX: Your GPU doesn't have enough memory. Reduce `batch_size` from 16 to 8 or 4 in the `DataLoader`. Or train on CPU (much slower but works).

■ ERROR: Error loading model: FileNotFoundError: rf_model.pkl

■ FIX: You haven't trained and saved the model yet. Run `ml/03_baseline_model.ipynb` completely first. Check that the file exists in `backend/models/`

■ ERROR: npm error: Cannot find module 'react-router-dom'

■ FIX: Dependencies not installed. Run: `cd frontend && npm install`

■ ERROR: Render deployment fails: No module named 'uvicorn'

■ FIX: `requirements.txt` is missing or has wrong path. Make sure Root Directory is set to 'backend' in Render settings.

■ ERROR: SHAP explainer gives wrong feature names

■ FIX: TF-IDF vectorizer and SHAP explainer must be from the SAME training run. Re-run training notebook from start.

■ ERROR: Image upload gives 422 Unprocessable Entity

■ FIX: Content-Type header must be `multipart/form-data`. Check `analyzeApi.js` — the Content-Type header in the `axios` config.

■ ERROR: BERT model takes too long to load

■ FIX: Normal — BERT is 400MB+. Load it once at startup (already done in `text_analyzer.py` as module-level variable). Do not reload per request.

■ ERROR: Accuracy is lower than 90% on first run

■ **FIX:** Check preprocessing — make sure `combined_text` is created correctly. Also check that ADASYN ran successfully. Run `accuracy_score()` and print the confusion matrix.

You now have the complete plan — every file, every line of code, every command. Follow each phase in order. Build. Test. Deploy. Present.

Target: 99%+ Accuracy | Live Deployed URL | Working Demo | Clean GitHub Repo